

Date: March 2026

Technical Documentation: Hands-On Email Threat Investigation

Prepared for: Career247 Cybersecurity Capstone Project

Prepared by: Mukul Kumar

Table of Contents

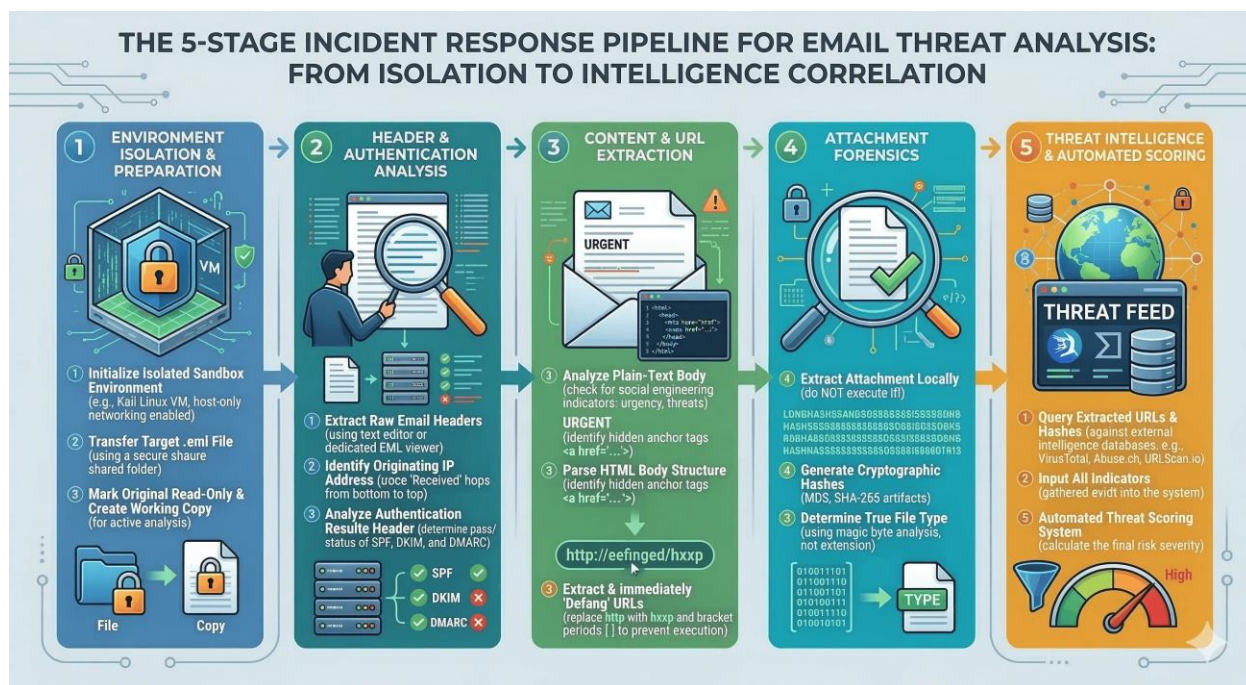
1. Reproducible Methodology for Future Analysts (Step-by-Step Workflow)
 - 1.1 Phase 1 – Environment Isolation & Preparation
 - 1.2 Phase 2 – Header & Authentication Analysis
 - 1.3 Phase 3 – Content & URL Extraction
 - 1.4 Phase 4 – Attachment Forensics
 - 1.5 Phase 5 – Threat Intelligence & Automated Scoring
2. Environment Setup and Tools Used
 - 2.1 Virtualization Platform
 - 2.2 Operating System
 - 2.3 Email Parsing Tools
 - 2.4 Command Line Utilities
 - 2.5 Threat Intelligence Platforms
 - 2.6 Automation & Development Stack
3. Commands Executed
 - 3.1 Kali Linux Forensics (Command Line Interface)
 - 3.2 Automated Scoring Deployment (Google Colab)
4. API Queries Performed
 - 4.1 VirusTotal URL Reputation Query
 - 4.2 VirusTotal File Hash Query
 - 4.3 Google GenAI Language Analysis
5. Screenshots of Critical Findings
 - 5.1 Automated Threat Scoring Dashboard
 - 5.2 External Intelligence – URL Analysis
 - 5.3 External Intelligence – Infrastructure & Payload Analysis
6. Challenges and Resolutions
 - 6.1 Safe File Handling and Defanging
 - 6.2 Hidden Malicious Destinations
 - 6.3 Interpreting Authentic Domain Spoofing

List of Figures

- Figure 1: Streamlit Threat Scoring Dashboard
- Figure 2: VirusTotal URL Analysis Result
- Figure 3: URLScan.io Infrastructure Analysis
- Figure 4: Abuse.ch Threat Intelligence Result
- Figure 5: YARAify Static Analysis Results

1. Reproducible Methodology for Future Analysts (Step-by-Step Workflow)

This methodology provides a standardized, repeatable workflow for safely triaging and investigating suspicious .eml files. Analysts should follow these steps linearly to ensure operational security and maintain the chain of custody.



Phase 1: Environment Isolation & Preparation

1. Initialize an isolated sandbox environment (e.g., Kali Linux VM) with host-only networking enabled.
2. Transfer the target .eml file into the sandbox using a secure shared folder.
3. Mark the original .eml file as read-only and create a duplicate working copy for active analysis.

Phase 2: Header & Authentication Analysis

1. Extract the raw email headers using a text editor or a dedicated EML viewer.
2. Identify the originating IP address by tracing the Received hops from bottom to top.
3. Analyze the Authentication-Results header to determine the pass/fail status of SPF, DKIM, and DMARC.

Phase 3: Content & URL Extraction

1. Analyze the plain-text body for social engineering indicators (e.g., urgency, threats).
2. Parse the HTML body structure to identify hidden anchor tags ().

3. Extract and immediately "defang" all discovered URLs (e.g., replace http with hxxp and bracket periods [.] to prevent accidental execution.

Phase 4: Attachment Forensics

1. Extract the attachment locally without executing it.
2. Generate cryptographic hashes (MD5, SHA-256) of the artifact.
3. Determine the true file type using magic byte analysis rather than relying on the file extension.

Phase 5: Threat Intelligence & Automated Scoring

1. Query extracted URLs and hashes against external intelligence databases (VirusTotal, Abuse.ch, URLScan.io).
2. Input all gathered indicators into the Automated Threat Scoring System to calculate the final risk severity.

2. Environment Setup and Tools Used

- **Virtualization:** VMware Workstation (Windows 11 Host).
- **Operating System:** Kali Linux (Isolated Guest OS).
- **Email Parsing:** Zoho Toolkit EML Viewer, MXToolBox.
- **Command Line Utilities:** ls, cd, cat, file, md5sum, sha256sum.
- **Threat Intelligence:** VirusTotal (GUI and API), Abuse.ch (URLhaus/ThreatFox), URLScan.io, YARAify.
- **Automation:** Google Colab, Python 3, Streamlit, Plotly, Google GenAI SDK, pyngrok.

3. Commands Executed

3.1 Kali Linux Forensics (CLI)

The following command-line operations were executed within the Kali Linux terminal to safely interact with and fingerprint the malicious attachment.

<Insert Infographic: Visual explanation of cryptographic hashing via CLI, demonstrating how tools like sha256sum create a unique digital fingerprint for malware identification.>

Bash

```
# Navigate to the working directory containing the extracted artifact
```

```
$ cd Downloads
```

```
# Verify the file's true format by analyzing its magic bytes, ignoring the .docx extension
```

```
$ file Invoice_99431.docx
# Output: Invoice_99431.docx: ASCII text, with no line terminators

# Safely output the contents of the ASCII text file to the terminal
$ cat Invoice_99431.docx
# Output: You got Hacked

# Generate the MD5 cryptographic hash for legacy intelligence correlation
$ md5sum Invoice_99431.docx
# Output: bfe751b0ae122a3305f35f420d43be65 Invoice_99431.docx

# Generate the SHA-256 cryptographic hash for modern intelligence correlation
$ sha256sum Invoice_99431.docx
# Output: 82bf93632480ca0c3e3856d671129bffe4910bd8a085e077afcb14ab0b12e7c
Invoice_99431.docx
```

3.2 Automated Scoring Deployment (Google Colab)

The following commands were utilized in the Google Colab notebook to provision the environment, bridge API secrets, and deploy the Streamlit threat analysis dashboard via an Ngrok tunnel.

Python

```
# Install required dependencies in the Colab environment
!pip install -q streamlit plotly google-genai python-dotenv pyngrok dnspython

# Write the Streamlit application logic to a local Python file
%%writefile app.py
import streamlit as st
import plotly.graph_objects as go
# ... [Full Application Code Truncated for Brevity] ...

# Bridge Colab secrets to Streamlit's native secrets management
import os
from google.colab import userdata
os.makedirs('.streamlit', exist_ok=True)
vt_key = userdata.get('VT_API_KEY')
with open('.streamlit/secrets.toml', 'w') as f:
    f.write(f'VT_API_KEY = "{vt_key}"\n')

# Deploy the Streamlit server and expose it to the web via an Ngrok tunnel
```

```
from pyngrok import ngrok
ngrok_token = userdata.get('NGROK_AUTH_TOKEN')
ngrok.set_auth_token(ngrok_token)

# Run Streamlit in the background pointing to localhost
os.system("nohup streamlit run app.py --server.port 8501 --server.address 127.0.0.1 > streamlit_logs.txt 2>&1 &")

# Establish the secure tunnel and output the public URL
public_url = ngrok.connect(addr="127.0.0.1:8501")
print(f"\n🌐 YOUR APP IS LIVE AT: {public_url.public_url}")
```

4. API Queries Performed

To automate the threat hunting process, custom Python functions were built to query external APIs within the Streamlit dashboard application.

<Insert Infographic: Flowchart detailing the request/response lifecycle between a Python security script and external Threat Intelligence APIs.>

1. VirusTotal URL Reputation Query (GET /api/v3/urls/{id})

- **Target:** <https://paypalphish.vercel.app/>
- **Encoding:** URL safely Base64 encoded.
- **Functionality:** Retrieved the last_analysis_stats to count how many security vendors flagged the URL as malicious.

2. VirusTotal File Hash Query (GET /api/v3/files/{hash})

- **Target:** 82bf93632480ca0c3e3856d671129bffe4910bd8a085e077afcb14ab0b12e7c
- **Functionality:** Cross-referenced the generated SHA-256 hash against known malware databases to identify payload signatures.

3. Google GenAI Language Analysis

- **Model:** gemini-2.5-flash
- **Prompt Structure:** Analyzed the Subject and Body of the email to evaluate the "Urgent or threatening language" factor on a scale of 0 to 10 and return a structured JSON threat summary.

5. Screenshots of Critical Findings

5.1 Automated Threat Scoring Dashboard

EMAIL THREAT ANALYZER

INPUT DATA

Upload .eml file for deep scan

Drag and drop file here
Limit 200MB per file • EML

Browse files

malmail.eml 2.4KB

✕

Sender Domain

secure-paypal.com

Email Subject

Action Required: Invoice #99431 Pending

Email Body

Dear Customer,

Your PayPal account has an unpaid invoice (invoice #99431). Please open the attached document to review your payment status.

URLs Found (One per line)

https://paypalphish.vercel.app/
https://paypal.com

Attachments Detected

Invoice_99431.docx

View Attachment Hashes (SHA-256)

Invoice_99431.docx
82bf93632480ca0c3e3856d671129bffe4910bd8a085e077afcb14ab0b12e7c

Authentication Status (Auto-Detected)

SPF
Fail

DKIM
None

DMARC
Fail

INITIATE SCAN

ANALYSIS RESULTS

THREAT LEVEL: SUSPICIOUS

36

Threat Vector Breakdown

VT Intel	15
Attach	0
URLs	3
Auth	10
Domain	0
AI Language	8

External Intel Summary

URL flagged by 12 vendors.

AI Threat Summary

This email employs strong urgent and threatening language, including 'Action Required' and a direct threat of 'account suspension,' to pressure the recipient. It attempts to impersonate PayPal to trick the user into opening a suspicious attachment or clicking a link. This is a phishing attempt, and users should not interact with the email but instead navigate directly to the official PayPal website to verify any account activity.

EXTERNAL THREAT INTELLIGENCE (VIRUSTOTAL)

Manual VirusTotal Scanner

Scan a URL

Enter URL to check
https://paypalphish.vercel.app/
Fetch URL Report

Scan a File Hash

Enter SHA-256 Hash
82bf93632480ca0c3e3856d671129bffe4910bd8a085e077afcb14ab0b12e7c
Fetch Hash Report

Figure 1: Custom Streamlit application deployed via Colab displaying the final calculated threat score of 36/100 (Suspicious) and the threat vector breakdown.

5.2 External Intelligence: URL Analysis

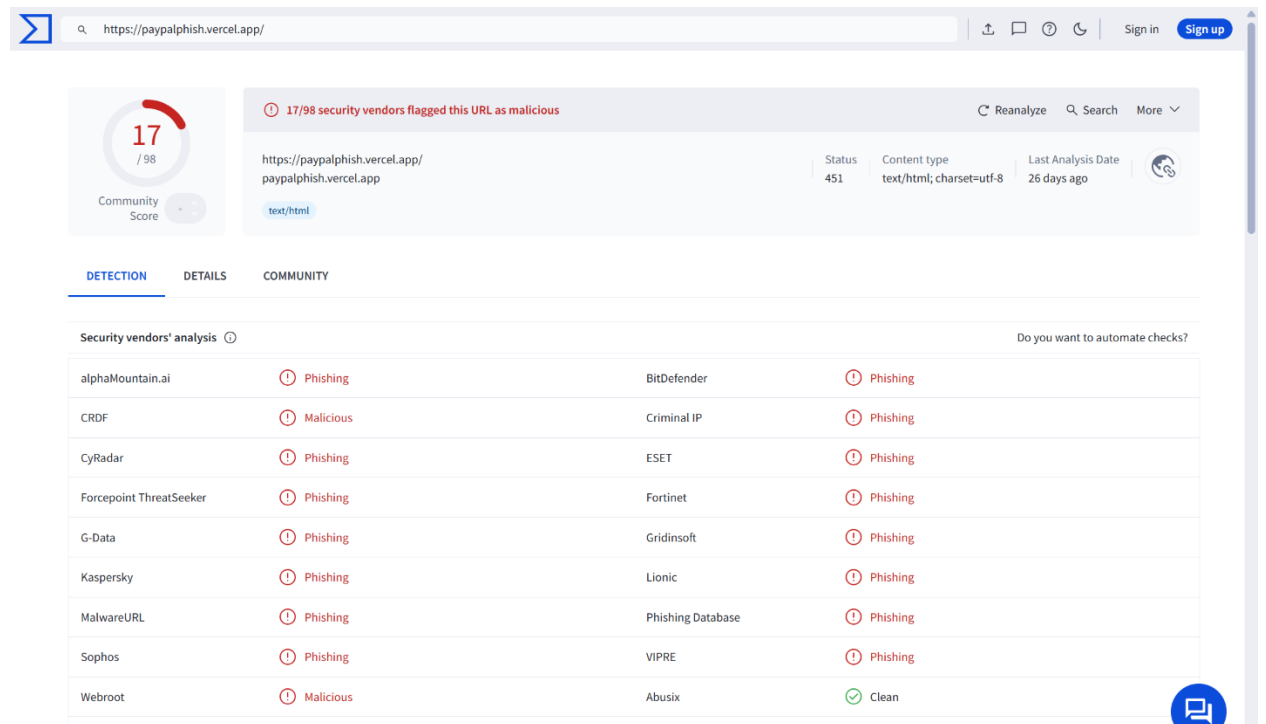


Figure 2: VirusTotal vendor analysis flagging the destination URL `paypalphish.vercel.app` as a phishing threat.

paypalphish.vercel.app
 216.198.79.3 **Malicious Activity!** Public Scan

URL: <https://paypalphish.vercel.app/>
 Submission: On October 09 via manual (October 9th 2025, 6:49:09 pm UTC) from IN - Scanned from US

Summary
 This website contacted 2 IPs in 1 countries across 1 domains to perform 2 HTTP transactions. The main IP is 216.198.79.3, located in United States and belongs to AMAZON-02, US. The main domain is paypalphish.vercel.app. TLS certificate: Issued by R13 on August 24th 2025. Valid for: 3 months.
 paypalphish.vercel.app scanned 27 times on urlscan.io Show Scans: 27
 urlscan.io Verdict: **Potentially Malicious** !
 Targeting these brands: PayPal (Financial)
 Live information
 Google Safe Browsing: **Malicious** for paypalphish.vercel.app
 Current DNS A record: 64.29.17.3 (AS16509 - AMAZON-02 - Amazon.com, Inc., US)

Screenshot
 Login - PayPal

Page Statistics

Requests	HTTPS	IPV6	Domains	Subdomains
2	100 %	0 %	1	1
IPs	Countries	Transfer	Size	Cookies
2	1	9 kB	28 kB	0

Domain & IP information

IP/ASNs	IP Detail	Domains	Domain Tree	Links	Certs	Frames
2	216.198.79.3	1	AS Autonomous System			
2		2	16509 (AMAZON-02)			

Figure 3: URLScan.io confirming the malicious domain is hosted on Vercel infrastructure (Amazon-02).

5.3 External Intelligence: Infrastructure & Payload Analysis

ABUSE.ch | **SPAMHAUS TECHNOLOGY** Hunting False Positives API Logout

Hunting Search Share

Summary URLhaus 23 MalwareBazaar ThreatFox YARAify Sandnet 22 IPintel 100 ProxyCheck FP

23 results found in our malware distribution URL tracking platform URLhaus.
22 result(s) found in Sandnet
100 result(s) found in our IP fingerprinting service IPintel.

Terms and Conditions | Terms of Use | Privacy Policy | Cookie Policy

Figure 4: Abuse.ch data confirming historical malicious activity tied to the 216.198.79.3

hosting IP.

YARAify Task Results

YARAify scan results for task ID **2124cec7-0fde-11f1-82f6-42010aa4000b**.

Scan Results

Task ID:	2124cec7-0fde-11f1-82f6-42010aa4000b		
Task parameters:	clamav_scan:	True	
	unpack:	True	
	share_file:	True	
Submission time:	2026-02-22 11:03:33 UTC		
Scan time:	Scan took 4 seconds		
File name:	mailmail.eml		
File size:	2'479 bytes		
File download:	 Original		
MIME type:	message/rfc822		
SHA256 hash:	 906f0d1e12173d562bd0b3cc4f3af94b1c6ec1ba443655b006bc0fe9d227dccc		
MD5 hash:	 a8a522ddae35390dcb32277b203ab1b3		
SHA1 hash:	 23a3cf6fb7ed2678ab49401c3991ec518a8d305d		
SHA3-384 hash:	 747e258fd66cde9c7da888a52f7d95dd35dd2e159dfe34f83a89de7aa4df1c4259f2240d501c4bdb2673b33a5dc90e8		
First seen:	2026-02-17 18:13:25 UTC		
Last seen:	2026-02-22 11:03:33 UTC		
Sightings:	2		
Imphash ⓘ:	n/a		
ssdeep ⓘ:	 48:6XyCUJGo6XREtxnX8n5AFVski/WAGH+bT4zLmdzlst:cyCUJGo6GnK5/rxSmlu		
TLSH ⓘ:	n/a		
telfhash ⓘ:	n/a		
dhash icon ⓘ:	n/a		

YARA Results

Static Analysis

The following YARA rules matched on the file (static analysis).

Rule name:	RANSOMWARE
Author:	ToroGuitar
TLP:	TLP:WHITE
Repository:	YARAify

Rule name:	with_urls
Author:	Antonio Sanchez <asanchez@hispasec.com>
Description:	Rule to detect the presence of an or several urls
Reference:	http://laboratorio.blogs.hispasec.com/
TLP:	TLP:WHITE

Unpacker

The following YARA rules matched on the **unpacked** file.

No matches

ClamAV Results

The file matched the following open source and commercial ClamAV rules.

No matches

Figure 5: YARAify task results verifying file hashes and running static analysis on the extracted .eml components.

6. Challenges and Resolutions

Challenge 1: Safe File Handling and Defanging

- **Issue:** Directly clicking links or opening the .docx file on a host machine risked executing a potential macro payload or confirming the target's active email status to the attacker.
- **Resolution:** Established an isolated Kali Linux VM. Used CLI tools (file, cat) instead of GUI word processors to inspect the attachment. All extracted URLs were immediately defanged in documentation to neutralize them.

Challenge 2: Hidden Malicious Destinations

- **Issue:** Initial scans of the plain-text email body only revealed a legitimate paypal.com URL, potentially resulting in a false negative.
- **Resolution:** Switched to raw HTML parsing. Inspected the anchor tags directly, which exposed the true redirection to the malicious vercel.app subdomain.

Challenge 3: Interpreting Authentic Domain Spoofing

- **Issue:** The "From" address appeared visually similar to a legitimate sender, which could easily bypass non-technical scrutiny.
- **Resolution:** Analyzed the Authentication-Results headers. Verified that the sending IP was not listed in the domain's SPF record and that the message lacked a DKIM signature, which ultimately triggered a DMARC failure.

7. Appendices

1. Installation of Dependencies

Python

```
!pip install -q streamlit plotly google-genai python-dotenv pyngrok dnspython
```

2. Application Code (app.py)

Python

```
%%writefile app.py
import streamlit as st
import plotly.graph_objects as go
from google import genai
import os
import json
import re
import hashlib
import requests
import base64
from email import policy
from email.parser import BytesParser
from dotenv import load_dotenv

# Optional DNS import for active domain checking
try:
    import dns.resolver
    DNS_AVAILABLE = True
except ImportError:
    DNS_AVAILABLE = False

# =====
# LOAD SECRETS
# =====
load_dotenv()
```

```

SYSTEM_API_KEY = os.getenv("GEMINI_API_KEY")

try:
    VT_API_KEY = st.secrets["VT_API_KEY"]
    VT_AVAILABLE = True
except (KeyError, FileNotFoundError):
    VT_API_KEY = None
    VT_AVAILABLE = False

# =====
# PAGE CONFIG & HACKER THEME CSS
# =====
st.set_page_config(page_title="Email Threat Analyzer", layout="wide")

st.markdown("""
<style>
.stApp { background-color: #0e1117; color: #00ff00; font-family: 'Courier New', Courier, monospace; }
h1, h2, h3 { color: #00ff00 !important; }
.stTextInput>div>div>input, .stTextArea>div>div>textarea, .stSelectbox>div>div>select {
    background-color: #1a1c23; color: #00ff00; border: 1px solid #00ff00;
    font-family: 'Courier New', Courier, monospace; border-radius: 6px; outline: none !important;
}
.stTextInput>div>div>input:focus, .stTextArea>div>div>textarea:focus,
.stSelectbox>div>div>select:focus {
    border-color: #00ff00 !important; box-shadow: 0 0 0 1px #00ff00 !important;
}
.stTextArea>div>div>textarea { resize: vertical; }
.stTextArea>div>div>textarea::-webkit-resizer { background-color: #1a1c23; }
.stTextInput>div>div>input:disabled { color: #00ff00; background-color: #0e1117; border: 1px dashed
#00ff00; }
.stButton>button { background-color: #00ff00; color: #0e1117; font-weight: bold; border: 2px solid
#00ff00; border-radius: 5px; }
.stButton>button:hover { background-color: #0e1117; color: #00ff00; border: 2px solid #00ff00; }
.expander-content { border: 1px solid #00ff00; padding: 10px; border-radius: 5px; background-color:
#1a1c23; }
</style>
""", unsafe_allow_html=True)

# =====
# VIRUSTOTAL API LOGIC
# =====
def check_file_hash(api_key, file_hash):
    url = f"https://www.virustotal.com/api/v3/files/{file_hash}"
    headers = {"accept": "application/json", "x-apikey": api_key}
    return requests.get(url, headers=headers).json()

def check_url_report(api_key, target_url):
    url_id = base64.urlsafe_b64encode(target_url.encode()).decode().strip("=")
    url = f"https://www.virustotal.com/api/v3/urls/{url_id}"

```

```

headers = {"accept": "application/json", "x-apikey": api_key}
return requests.get(url, headers=headers).json()

# =====
# CORE SCORING LOGIC (100 PT MATRIX)
# =====
DANGEROUS_EXTENSIONS = [
    '.exe', '.bat', '.cmd', '.com', '.pif', '.scr', '.vbs', '.js',
    '.jar', '.zip', '.rar', '.7z', '.iso', '.dll', '.sys', '.msi',
    '.hta', '.ps1', '.psm1', '.reg', '.scf', '.lnk', '.inf'
]

def calculate_attachment_score(attachments_input):
    if not attachments_input: return 0
    score = sum(10 for att in (a.strip().lower() for a in attachments_input.split(','))
                if any(att.endswith(ext) for ext in DANGEROUS_EXTENSIONS))
    return min(score, 30)

```